

IOT · CONDIÇÃO DE EQUIPAMENTO · N1

Aquário Inteligente

Monitoramento de pH com ESP32, MQTT e atuador comandado pela rede

Gabrieli Eduarda Lembeck · Heloisa Rebello Cabral · Julio Bezerra de Mattos Manoel
Mileine da Silva de Freitas · Thomas Henry Steinback

O problema

**Medir o pH do aquário à mão
deixa variações passarem
despercebidas até virarem um
problema para os peixes.**

A proposta

Um sensor ligado ao ESP32 mede o pH continuamente e envia cada leitura pela rede.

Um servidor avalia o valor e devolve um comando que aciona o alerta no próprio dispositivo.

A solução em três papéis

1

ESP32 lê

Mede o pH e publica a leitura via Wi-Fi em um tópico MQTT.

2

Servidor decide

Assina o tópico, compara com a faixa 6,5–7,5 e publica ON ou OFF.

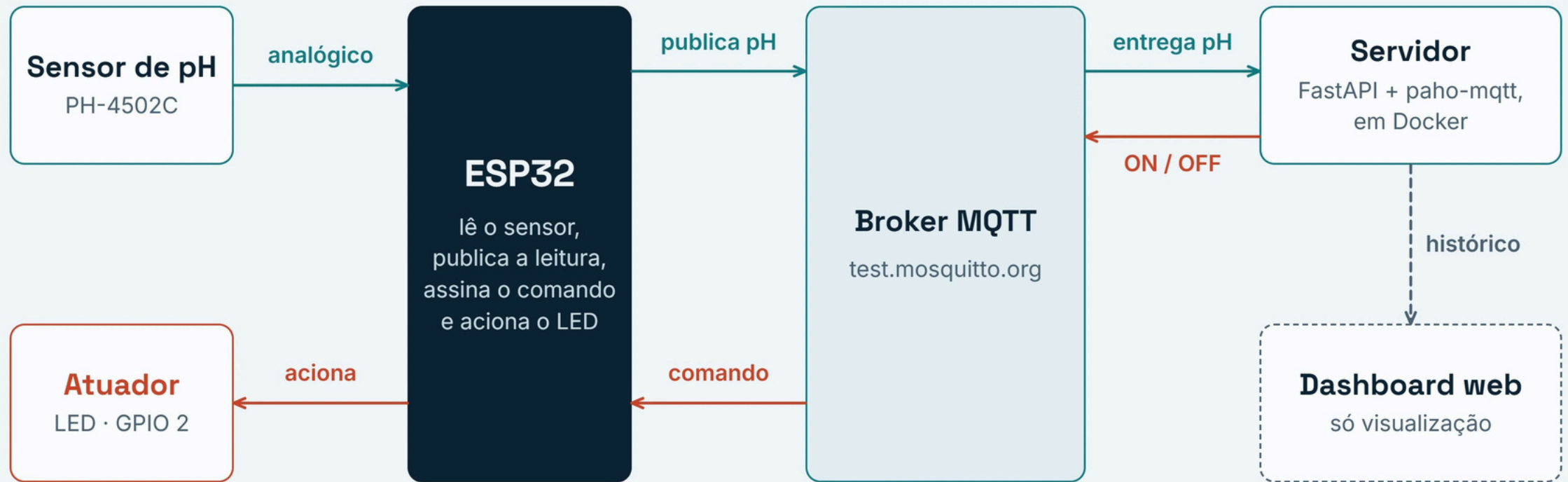
3

ESP32 executa

Recebe o comando pelo MQTT e aciona o atuador (LED no GPIO 2).

O firmware **não decide nada** sobre o atuador: sem rede e sem broker, ele não muda de estado.

Arquitetura: o caminho da leitura e do comando



Em paralelo, o ESP32 envia cada leitura por HTTP POST ao servidor só para alimentar o dashboard. O comando do atuador passa apenas pelo MQTT.

Diagrama em blocos do hardware



Alimentação do módulo de pH: [3,3 V ou 5 V — confirmar na montagem]. Em 5 V, usar divisor 10k/20k no pino Po e FATOR_DIVISOR = 1,5.

Componentes

Componente	Função	Ligação
ESP32 DevKit	Lê o sensor, publica e assina MQTT, aciona o atuador	USB e Wi-Fi
Sensor de pH PH-4502C	Converte o pH da água em tensão analógica	Po → GPIO 35 (ADC)
Display OLED SSD1306	Mostra Wi-Fi, tensão, pH e estado do atuador	I2C: SDA 21, SCL 22
LED embutido	Atuador: acende só por comando MQTT	GPIO 2
Servidor FastAPI	Assina o pH, decide ON/OFF e serve o dashboard	Docker na rede local
Broker MQTT	Intermedia as mensagens (test.mosquitto.org)	1883 no ESP32 · 8883 com TLS no servidor

Tópicos MQTT

Tópico	Publica	Assina	Conteúdo
sistema/aquario/ph	ESP32	Servidor	{"ph":7.02,"tensao":2.513}
sistema/aquario/atuador	Servidor	ESP32	ON ou OFF (mensagem retida)

Prefixo único

sistema/aquario/ evita colisão com outras equipes no broker público.

Só na mudança

O servidor publica o comando apenas quando o estado muda, com QoS 1.

Retain

Ao reconectar, o ESP32 recebe na hora o último comando enviado.

Funcionamento: um ciclo por segundo

1 • Leitura

15 amostras do GPIO 35 a cada 1 s, sem travar o loop (millis).

2 • Filtro

Ordena, descarta as 3 menores e as 3 maiores e faz a média das 9 centrais.

3 • Conversão

$\text{pH} = -5,70 \times \text{tensão} + 21,34$, com aviso se o ADC saturar.

4 • Publicação

Envia o pH em sistema/aquario/ph e uma cópia por HTTP ao dashboard.

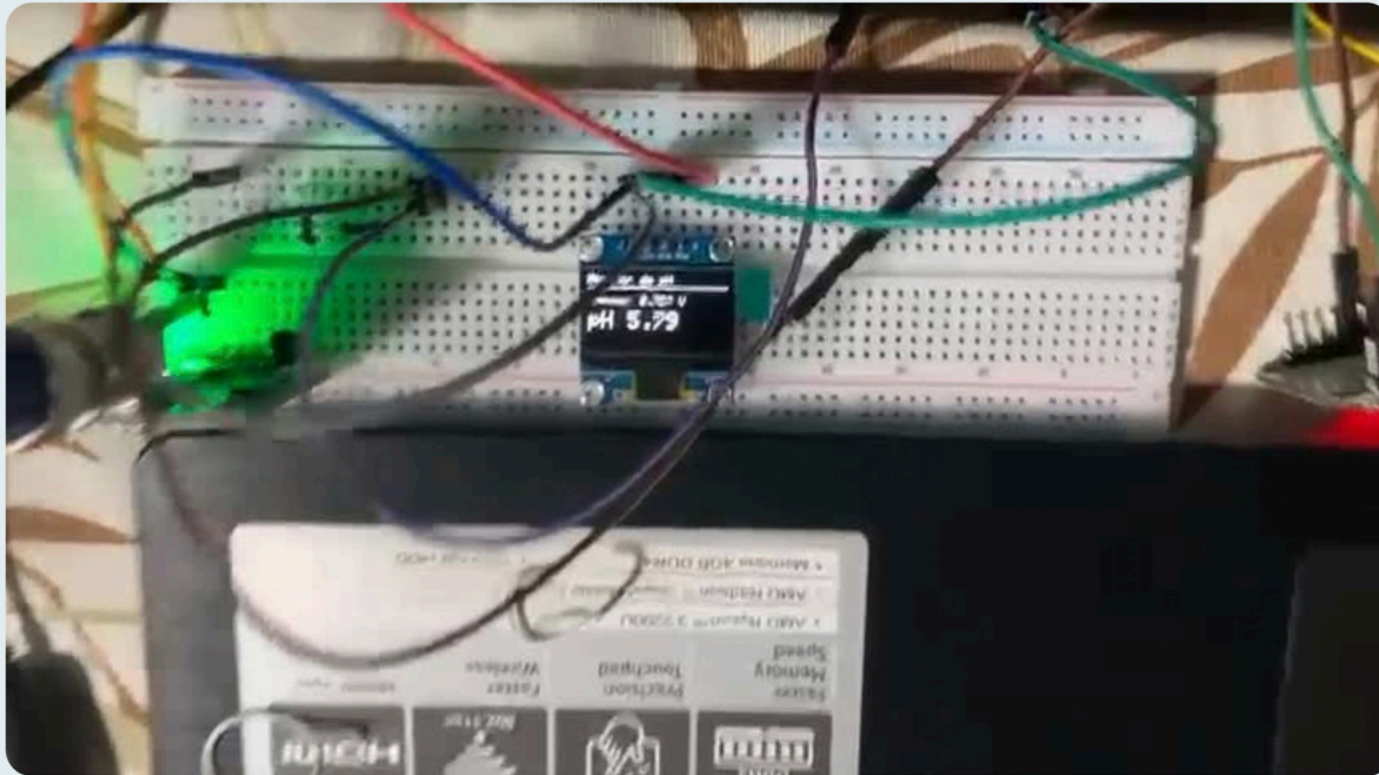
5 • Decisão

O servidor compara com 6,5–7,5 e publica ON ou OFF no tópico do atuador.

6 • Atuação

O callback MQTT liga ou desliga o LED e atualiza o OLED.

Protótipo em bancada



Montagem

ESP32, módulo PH-4502C e OLED na protoboard.

Leitura real

O OLED mostra a tensão e o pH em tempo real.

Atuador

LED embutido da placa, sem ligação extra.

Código: leitura filtrada do sensor

```
float lerTensaoFiltrada() {  
    for (int i = 0; i < NUM_SAMPLES; i++) { // 15 leituras  
        buffer_arr[i] = analogRead(PH_PIN);  
        delay(20);  
    }  
    // ordenação simples por troca (crescente)  
    // ...  
    int inicio = NUM_SAMPLES / 4; // descarta 3 por ponta  
    int fim     = NUM_SAMPLES - inicio;  
    for (int i = inicio; i < fim; i++) soma += buffer_arr[i];  
    float leituraMedia = (float)soma / (fim - inicio);  
    adcSaturado = leituraMedia ≥ ADC_SATURADO;  
    return (leituraMedia * 3.3) / 4095.0 * FATOR_DIVISOR;  
}
```

$$\text{pH} = -5,70 \times V + 21,34$$

Ruído elétrico

Picos isolados vão para as pontas e são descartados.

Média aparada

Usa 9 das 15 amostras.

Saturação

Média ≥ 4090 gera aviso no Serial.

Código: publicar a leitura e assinar o comando

reconectarMQTT() · assina o comando

```
void reconectarMQTT() {
    if (WiFi.status() != WL_CONNECTED) return;
    if (client.connected()) return;
    unsigned long agora = millis();
    if (agora - ultimaTentativaMQTT < 2000)
        return; // não trava o loop
    ultimaTentativaMQTT = agora;
    String clientId = "ESP32-pH-" +
        String(random(0xffff), HEX);
    if (client.connect(clientId.c_str()))
        client.subscribe(topico_atuador);
}
```

publicarMQTT() · envia a leitura

```
void publicarMQTT(float tensao, float ph) {
    if (!client.connected()) return;
    String payload = "{\"ph\":\"" + String(ph, 2)
        + "\",\"tensao\":\"" + String(tensao, 3) + "\"}";
    client.publish(topico_ph, payload.c_str());
}

// tópico: sistema/aquario/ph
// exemplo: {"ph":7.02,"tensao":2.513}
```

Código: o callback que aciona o atuador

```
void aoReceberMensagem(char* topico, byte* payload,
                      unsigned int tamanho) {
    String mensagem;
    for (unsigned int i = 0; i < tamanho; i++)
        mensagem += (char)payload[i];
    mensagem.trim(); mensagem.toUpperCase();
    if (String(topico) != topico_atuador) return;
    if (mensagem == "ON") {
        atuadorLigado = true;
        digitalWrite(ATUADOR_PIN, HIGH);
    } else if (mensagem == "OFF") {
        atuadorLigado = false;
        digitalWrite(ATUADOR_PIN, LOW);
    } else return; // comando desconhecido: ignora
    mostrarNoOLED(ultimaTensao, ultimoPh);
}
```

Único ponto do firmware que mexe no atuador

Registro

No setup():
client.setCallback(aoReceberMensagem).

Entrada aceita

Apenas ON e OFF no tópico do atuador.

Estado inicial

LED começa desligado no setup().

Código: o loop principal

```
void loop() {  
  if (WiFi.status() != WL_CONNECTED) conectarWiFi();  
  reconectarMQTT();  
  client.loop(); // processa comandos recebidos  
  
  unsigned long agora = millis();  
  if (agora - ultimaLeitura < INTERVALO_LEITURA_MS) return;  
  ultimaLeitura = agora;  
  
  float tensao = lerTensaoFiltrada();  
  float ph = PH_SLOPE * tensao + PH_OFFSET;  
  
  publicarMQTT(tensao, ph); // 1) leitura sai via MQTT  
  enviarLeitura(ph, tensao); // 2) cópia HTTP  
  mostrarNoOLED(tensao, ph);  
} // nenhuma decisão sobre o atuador aqui
```

Sem delay()

O loop volta rápido, e client.loop() recebe comandos a qualquer momento.

Periodicidade

Uma leitura a cada 1 s, controlada por millis().

Reconexão

Wi-Fi e MQTT voltam sozinhos se a conexão cair.

Sem automação local

O loop só lê e publica; quem decide é o servidor.

Servidor: onde a decisão acontece

```
def ao_receber(client, userdata, msg):  
    dados = json.loads(msg.payload.decode("utf-8"))  
    ph = float(dados["ph"])  
  
    comando = "ON" if (ph < PH_MIN or ph > PH_MAX) else "OFF"  
  
    # publica só quando o estado muda  
    if comando != estado["atuador"]:  
        client.publish(TOPICO_ATUADOR, comando,  
                        qos=1, retain=True)  
        estado["atuador"] = comando
```

Faixa normal

pH 6,5 a 7,5, configurável no docker-compose.yml.

MQTT em paralelo à API

O cliente paho-mqtt roda em thread própria, junto com o FastAPI.

Mensagem inválida

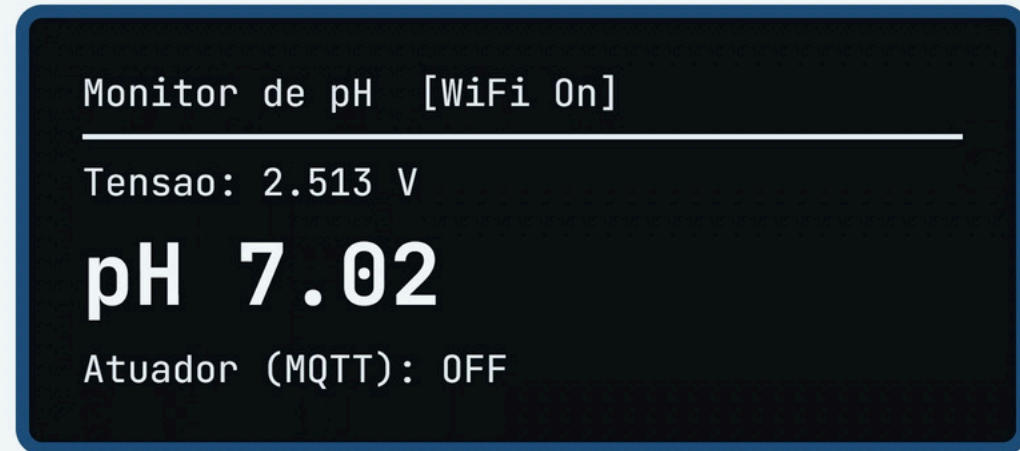
JSON sem pH é registrado no log e descartado.

retain + QoS 1

O comando chega mesmo se o ESP32 reconectar depois.

Saídas: display local e dashboard web

Display OLED



Representação da tela com valores de exemplo; o estado do atuador vem do último comando MQTT.

Dashboard web

`http://<IP>:8000`

pH atual, faixa em que está e o comando do atuador.

`/api/status`

Conexão MQTT, último pH recebido e último comando.

`/api/ph/historico`

As últimas 50 leituras recebidas por HTTP.

Demonstração do requisito de IoT

1

pH fora da faixa

Sensor numa solução fora de 6,5–7,5: o servidor publica ON e o LED acende.

2

Servidor parado

Com docker compose stop, o pH segue sendo lido, mas o LED não muda mais.

3

Tópicos ao vivo

```
mosquitto_sub -h  
test.mosquitto.org -t  
"sistema/aquario/#" -v
```

O passo 2 prova que o comando vem da rede, não do firmware.

Riscos técnicos e mitigação

RISCO PRINCIPAL

O broker público pode ficar instável ou fora do ar na apresentação.

Mitigação: prefixo único nos tópicos, comando como mensagem retida e troca por um Mosquitto local alterando mqtt_server no firmware e MQTT_BROKER no docker-compose.yml.

RISCO DE HARDWARE

Acima de ~3,1 V o ADC satura e o pH fica travado.

Mitigação: detecção de saturação no código e divisor 10k/20k no pino Po quando o módulo for alimentado em 5 V.

Obrigado!

Perguntas?

github.com/gabilbck/IoT-CondicaoDeEquipamento

